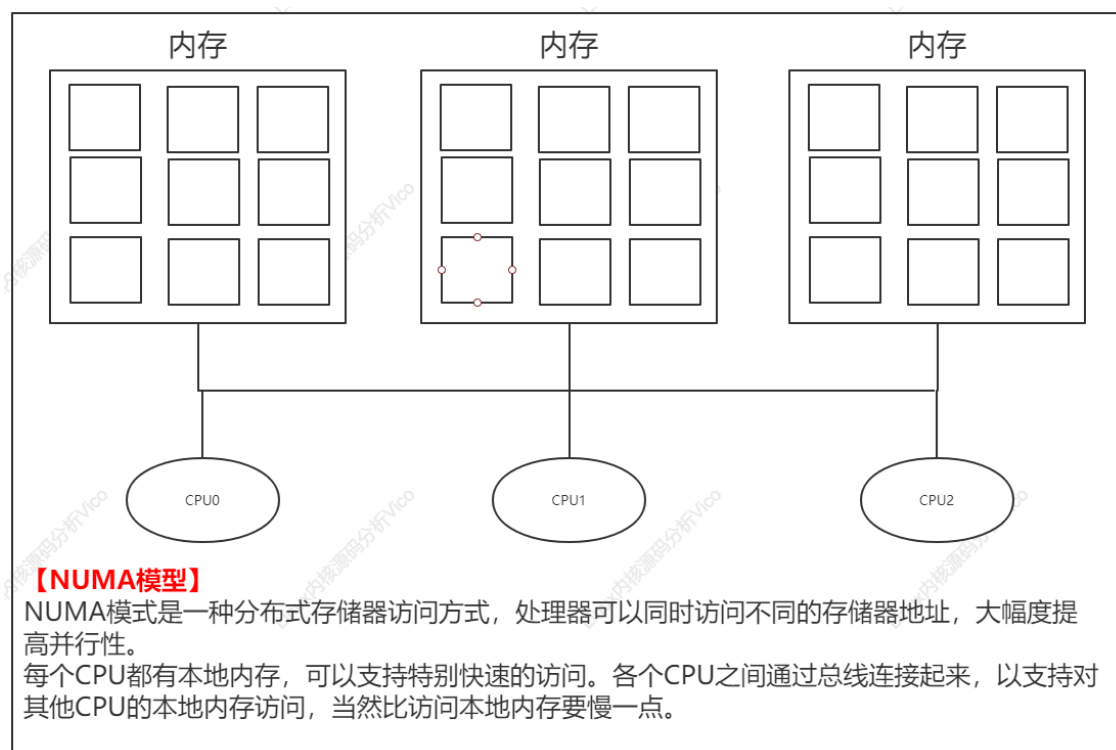
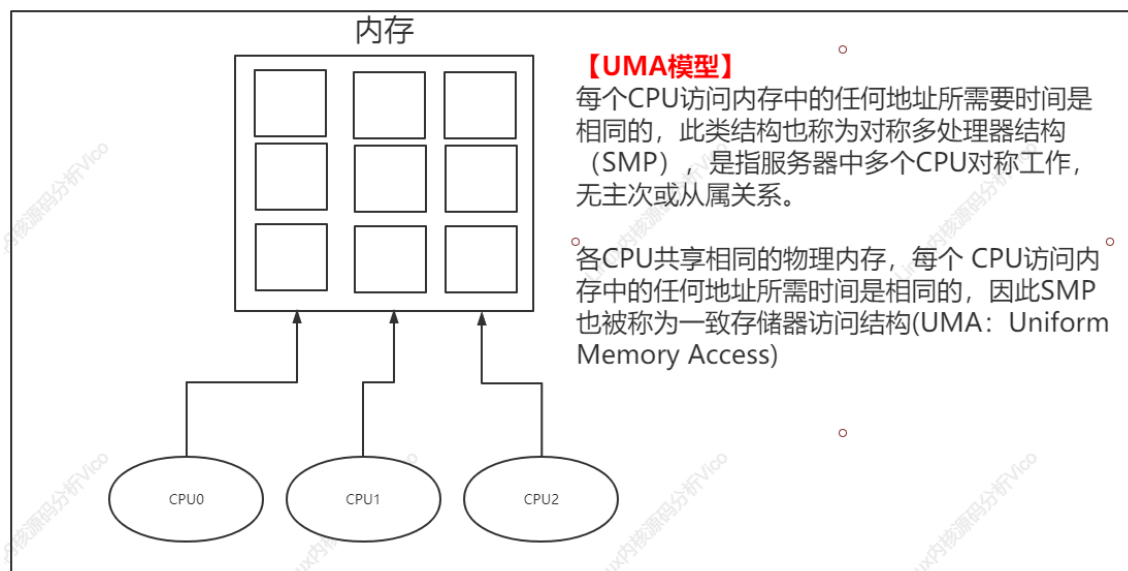


一、UMA/NUMA内存结构

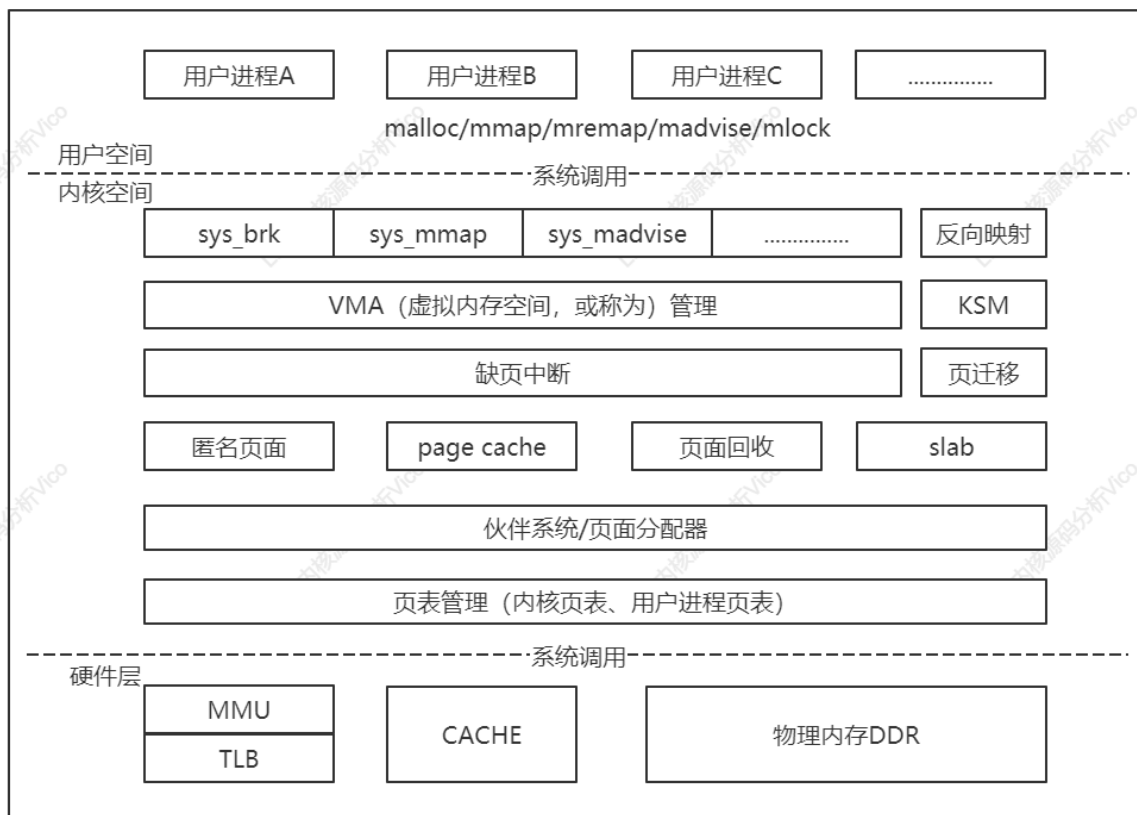
共享存储型多处理机有两种模型：

- 均匀存储器存取（Uniform-Memory-Access，简称UMA）模型
- 非均匀存储器存取（Nonuniform-Memory-Access，简称NUMA）模型



二、mm_struct结构体

Linux内核内存管理子系统架构如下图所示，分为用户空间、内核空间和硬件层3个层面。



1、用户空间

应用程序使用`malloc()`申请内存，使用`free()`释放内存，`malloc()/free()`是glibc库的内存分配器`ptmalloc`提供的接口，`ptmalloc`使用系统调用`brk/mmap`向内核以页为单位申请内存，然后划分成小内存块分配给用户应用程序。

2、内核空间

内核空间的基本功能：虚拟内存管理负责从进程的虚拟地址空间分配虚拟页，`sys_brk`用来扩大或收缩堆，`sys_mmap`用来在内存映射区域分配虚拟页，`sys_munmap`用来释放虚拟页。

内核空间的扩展功能：不连续页分配器提供了分配内存的接口`vmalloc`和释放内存接口`vfree`，在内存碎片化时，申请连续物理页的成功率很低，可申请不连续的物理页，映射到连续的虚拟地址，即虚拟地址连续页物理地址不连续。

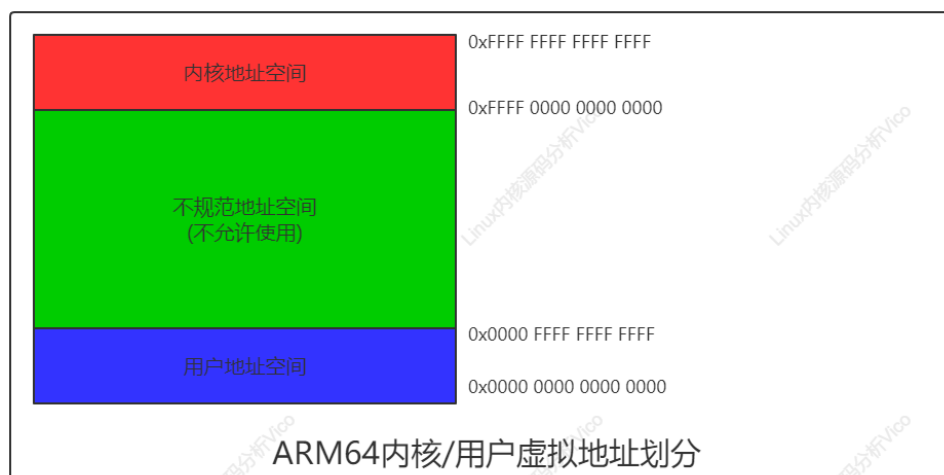
3、硬件层面

处理器包含一个称为内存管理单元（Memory Management Unit, MMU）的部件，负责把虚拟地址转换成物理地址。内存管理单元包含一个称为大页表缓存

(Translation Lookaside Buffer, TLB) 的部件，保存最近使用的页表映射，避免每次把虚拟地址转换物理地址都需要查询内存中的页表。

4、虚拟地址空间布局

a、虚拟地址空间划分



b、用户虚拟地址空间布局

进程的用户虚拟地址空间的起始地址是0，长度是TASK_SIZE，由每种处理器架构定义自己的宏TASK_SIZE。ARM64架构定义宏TASK_SIZE如下所示：

- 32位用户空间程序：TASK_SIZE的值是TASK_SIZE_32，即0x10000000，等于4GB。
- 64位用户空间程序：TASK_SIZE的值是TASK_SIZE_64，即2的VA_BITS次方字节，VA_BITS是编译内核时选择的虚拟地址位数。

```
arch > arm64 > include > asm > C memory.h > ...  
68 #define VA_BITS (CONFIG_ARM64_VA_BITS)  
69 #define VA_START (UL(0xffffffffffffffff) << VA_BITS)  
70 #define PAGE_OFFSET (UL(0xffffffffffffffff) << (VA_BITS - 1))  
71 #define KIMAGE_VADDR (MODULES_END)  
72 #define MODULES_END (MODULES_VADDR + MODULES_VSIZE)  
73 #define MODULES_VADDR (VA_START + KASAN_SHADOW_SIZE)  
74 #define MODULES_VSIZE (SZ_128M)  
75 #define VMEMMAP_START (PAGE_OFFSET - VMEMMAP_SIZE)  
76 #define PCI_IO_END (VMEMMAP_START - SZ_2M)  
77 #define PCI_IO_START (PCI_IO_END - PCI_IO_SIZE)  
78 #define FIXADDR_TOP (PCI_IO_START - SZ_2M)  
79 #define TASK_SIZE_64 (UL(1) << VA_BITS)  
80  
81 #ifdef CONFIG_COMPAT  
82 #define TASK_SIZE_32 UL(0x100000000)
```

Linux内核使用内存描述符mm_struct描述进程的用户虚拟地址空间，主要核心成员如下：

```
include > linux > C mm_types.h > 55 kiocx_table
361
362 struct mm_struct {
363     struct vm_area_struct *mmap;          /* list of VMAs */
364     struct rb_root mm_rb;
365     u32 vmacache_seqnum;                  /* per-thread vmacache */
```

```
struct mm_struct {
    struct vm_area_struct *mmap;          /* list of VMAs */
    struct rb_root mm_rb;
    u32 vmacache_seqnum;                  /* per-thread vmacache */
```

```
#ifdef CONFIG_MMU
```

```
    unsigned long (*get_unmapped_area) (struct file *filp,
        unsigned long addr, unsigned long len,
        unsigned long pgoff, unsigned long flags);
```

```
#endif
```

```
    unsigned long mmap_base;              /* base of mmap area */
    unsigned long mmap_legacy_base;        /* base of mmap area in botto
m-up allocations */
```

```
#ifdef CONFIG_HAVE_ARCH_COMPAT_MMAP_BASES
```

```
    /* Base addresses for compatible mmap() */
    unsigned long mmap_compat_base;
    unsigned long mmap_compat_legacy_base;
```

```
#endif
```

```
    unsigned long task_size;              /* size of task vm space */
    unsigned long highest_vm_end;          /* highest vma end address */
    pgd_t * pgd;
    /**
```

```
    * @mm_users: The number of users including userspace.
    *
```

```

* Use mmget()/mmget_not_zero()/mmapput() to modify. When this drop
s
* to 0 (i.e. when the task exits and there are no other temporary
* reference holders), we also release a reference on @mm_count
* (which may then free the &struct mm_struct if @mm_count also
* drops to 0).
*/
atomic_t mm_users;
/**
* @mm_count: The number of references to &struct mm_struct
* (@mm_users count as 1).
*
* Use mmgrab()/mmdrop() to modify. When this drops to 0, the
* &struct mm_struct is freed.
*/
atomic_t mm_count;
atomic_long_t nr_ptes;      /* PTE page table pages */
#ifdef CONFIG_PGTABLE_LEVELS > 2
atomic_long_t nr_pmds;      /* PMD page table pages */
#endif
int map_count;              /* number of VMAs */
spinlock_t page_table_lock; /* Protects page tables and some count
ers */
struct rw_semaphore mmap_sem;
struct list_head mmlist;    /* List of maybe swapped mm's. These are
globally strung
    * together off init_mm.mmlist, and are protected
    * by mmlist_lock
    */

```

```

unsigned long hiwater_rss; /* High-watermark of RSS usage */
unsigned long hiwater_vm; /* High-water virtual memory usage */
unsigned long total_vm; /* Total pages mapped */
unsigned long locked_vm; /* Pages that have PG_mlocked set */
unsigned long pinned_vm; /* Refcount permanently increased */
unsigned long data_vm; /* VM_WRITE & ~VM_SHARED & ~VM_ST
ACK */
unsigned long exec_vm; /* VM_EXEC & ~VM_WRITE & ~VM_STACK
*/
unsigned long stack_vm; /* VM_STACK */
unsigned long def_flags;
unsigned long start_code, end_code, start_data, end_data;
unsigned long start_brk, brk, start_stack;
unsigned long arg_start, arg_end, env_start, env_end;
unsigned long saved_auxv[AT_VECTOR_SIZE]; /* for /proc/PID/auxv */
/*
 * Special counters, in some configurations protected by the
 * page_table_lock, in other configurations by being atomic.
 */
struct mm_rss_stat rss_stat;
struct linux_binfmt *binfmt;
cpumask_var_t cpu_vm_mask_var;
/* Architecture-specific MM context */
mm_context_t context;
unsigned long flags; /* Must use atomic bitops to access the bits */
struct core_state *core_state; /* coredumping support */
#ifdef CONFIG_AIO
spinlock_t ioctx_lock;
struct kiocx_table __rcu *ioctx_table;

```

```
#endif
```

```
#ifdef CONFIG_MEMCG
```

```
/*
```

```
 * "owner" points to a task that is regarded as the canonical
 * user/owner of this mm. All of the following must be true in
 * order for it to be changed:
```

```
 *
```

```
 * current == mm->owner
```

```
 * current->mm != mm
```

```
 * new_owner->mm == mm
```

```
 * new_owner->alloc_lock is held
```

```
*/
```

```
struct task_struct __rcu *owner;
```

```
#endif
```

```
struct user_namespace *user_ns;
```

```
/* store ref to file /proc/<pid>/exe symlink points to */
```

```
struct file __rcu *exe_file;
```

```
#ifdef CONFIG_MMU_NOTIFIER
```

```
struct mmu_notifier_mm *mmu_notifier_mm;
```

```
#endif
```

```
#if defined(CONFIG_TRANSPARENT_HUGEPAGE) && !USE_SPLIT_PMD_P  
TLOCKS
```

```
pgtable_t pmd_huge_pte; /* protected by page_table_lock */
```

```
#endif
```

```
#ifdef CONFIG_CPUMASK_OFFSTACK
```

```
struct cpumask cpumask_allocation;
```

```
#endif
```

```
#ifdef CONFIG_NUMA_BALANCING
```

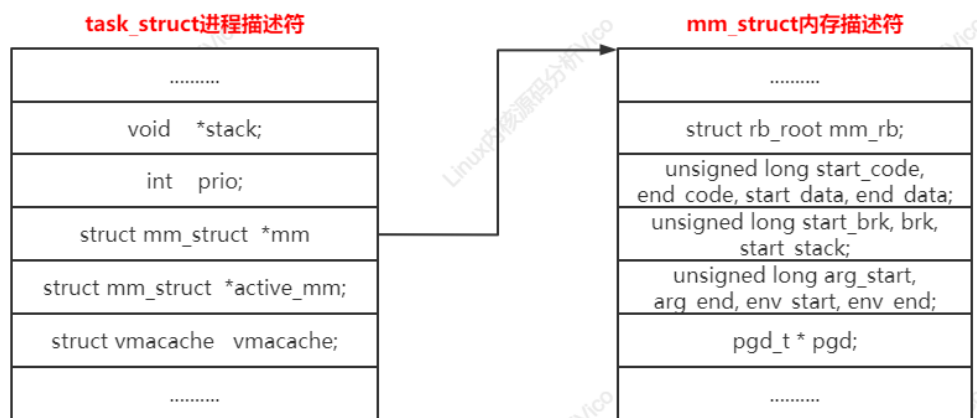
```
/*
```

```

* numa_next_scan is the next time that the PTEs will be marked
* pte_numa. NUMA hinting faults will gather statistics and migrate
* pages to new nodes if necessary.
*/
unsigned long numa_next_scan;
/* Restart point for scanning and setting pte_numa */
unsigned long numa_scan_offset;
/* numa_scan_seq prevents two threads setting pte_numa */
int numa_scan_seq;
#endif
#if defined(CONFIG_NUMA_BALANCING) || defined(CONFIG_COMPACTION)
/*
 * An operation with batched TLB flushing is going on. Anything that
 * can move process memory needs to flush the TLB when moving a
 * PROT_NONE or PROT_NUMA mapped page.
 */
bool tlb_flush_pending;
#endif
    struct uprobes_state uprobes_state;
#ifdef CONFIG_HUGETLB_PAGE
    atomic_long_t hugetlb_usage;
#endif
    struct work_struct async_put_work;
};

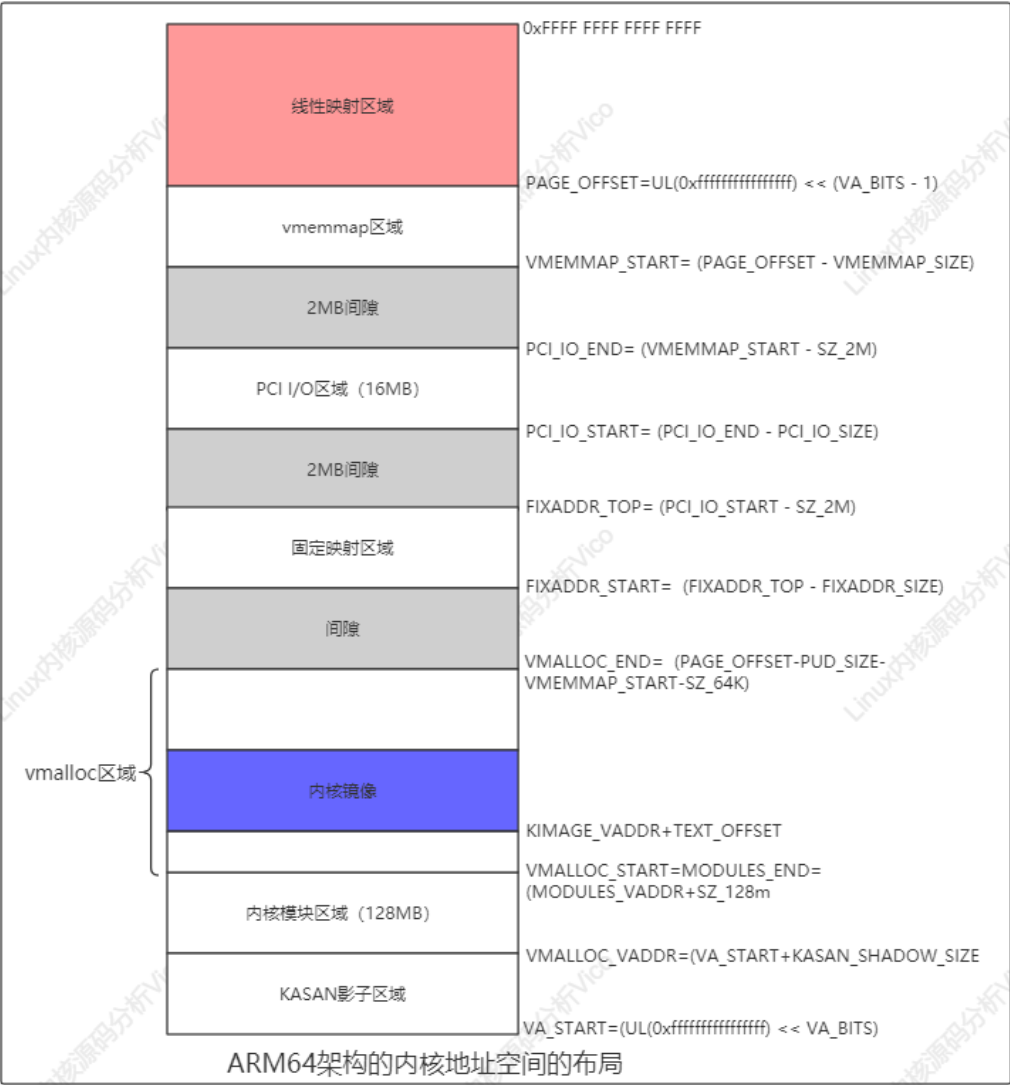
```

c、进程的进程描述和内存描述符关系如下图所示：



c、内核地址空间布局

ARM64处理器架构的内核地址空间布局如下：



三、TLB工作原理

处理器的内存管理单元（Memory Management Unit，MMU）负责把虚拟地址转换成物理地址，为了改进虚拟地址到物理地址的转换速度，避免每次转换都需要查找内存中的页表，处理器厂商在内存管理单元里面增加一个称为TLB

(Translation Lookaside Buffer, TLB) 的高速缓存, TLB直接为转换后备缓冲区, 意译为页表缓存。

1、TLB表项格式

不同处理器架构的TLB表项的格式不同, ARM64处理器的每条TLB表项不仅包含虚拟地址和物理地址, 也包含属性: 内存类型、缓存策略、访问权限、地址空间标识符 (ASID) 和虚拟机标识符 (VMID) 。

2、TLB管理

如果内核修改了可能缓存在TLB里面的页表项, 那么内核必须负责使旧的TLB表项失效, 内核定义每种处理器架构必须实现的函数如下:

```
arch > arm64 > include > asm > C tlbflush.h > _ASM_TLBFLUSH_H
103
104 static inline void flush_tlb_all(void)
```

```
static inline void flush_tlb_all(void)
```

```
static inline void flush_tlb_mm(struct mm_struct *mm)
```

```
static inline void flush_tlb_range(struct vm_area_struct *vma, unsigned long start, unsigned long end)
```

```
static inline void flush_tlb_page(struct vm_area_struct *vma, unsigned long uaddr)
```

```
static inline void flush_tlb_kernel_range(unsigned long start, unsigned long end)
```

3、ARM64架构TLB失效指令: TLB<type><level>[IS]{,<xt>}

flush_tlb_all用来使所有核的所有TLB失效, 内核代码如下:

```
arch > arm64 > include > asm > C tlbflush.h > flush_tlb_page(vm_area_struct *, unsigned long)
104 static inline void flush_tlb_all(void)
105 {
106     dsb(ishst);
107     __tlbi(vmlle1is);
108     dsb(ish);
109     isb();
110 }
```

四、页表

层次化的页表用于支持对大地址空间的快速、高效的管理。页表用于建立用户进程的虚拟地址空间和系统物理内存（内存、页帧）之间的关联。页表用来把虚拟页映射到物理页，并且存放页的保护位，即访问权限。Linux内核把页表分为4级：PGD、PUD、PMD、PT。